

Personal Expenses and Budget Tracker

Individual Assignment 1

1. Introduction:

The Personal Expense & Budget Tracker is a small monolithic web application developed using Python, Flask and SQLite. The purpose of the application is to allow a user to record personal expenses and create monthly spending budgets. The application is divided into two main feature domains: Expenses and Budgets.

The Expense domain allows the user to enter an expense with a description, amount, category and date. The Budget domain allows the user to set a monthly spending limit for a category and compare that budget with expenses recorded in the same category and month. The application then calculates the amount spent and the remaining budget.

The application was intentionally kept simple so that its structure and business logic remain understandable while still providing two distinct backend domains. It currently runs as a single Flask process and uses a single SQLite database.

2. SMART goals:

Goal 1 — Expense Tracking: By the assignment deadline, implement an Expense domain that allows a user to record expenses containing a description, positive amount, category and date, store them in SQLite, and display the stored expenses through the website interface.

Goal 2 — Budget Tracking: By the assignment deadline, implement a Budget domain that allows a user to set a monthly budget for a category and automatically calculate the amount spent and remaining based on the recorded expenses.

Goal 3 — Automated Testing: Before submission, create automated tests for the core business logic of both domains and achieve at least 70% code coverage. The final application achieved 92% coverage with eight passing tests.

Goal 4 — Deployment Readiness: Before submission, configure the application so that it can start using one command, bind to `0.0.0.0`, use a configurable `PORT`, and store its SQLite database in a configurable `DATA_DIR`.

3. Software Development Life Cycle

I developed the application step by step instead of trying to build everything at once. I followed the main stages of planning, design, implementation, testing and documentation.

Planning: I first decided what type of application I wanted to build and what features it needed. I chose a Personal Expense & Budget Tracker because it gave me two clear domains, Expenses and Budgets, while still being simple enough for a small application.

Design: I decided to build the application using Flask and SQLite. I separated the Expense and Budget logic into different Python files so that each domain had its own responsibility. I also created separate *expenses* and *budgets* tables in the database.

Implementation: I started by setting up Flask and the SQLite database. I then built the Expense feature first, followed by the Budget feature. After the main features worked, I added validation and made it possible to update a budget if the same category and month already existed.

Testing: I used pytest to test the main logic of both domains. The tests check things such as invalid amounts, saving data to the database, calculating spending and calculating the remaining budget. I also used pytest-cov to check the code coverage. The final result was eight passing tests with 92% total coverage.

Documentation and Deployment Preparation: Finally, I added the README, ADRs and AI usage log. I also changed the application so the port and database directory can be controlled using environment variables. The application binds to *0.0.0.0*, which makes it more suitable for deployment later

4. Application Architecture

The application uses a simple monolithic architecture, meaning everything runs as one application. The browser sends requests to the Flask application, and *app.py* handles the routes. The Expense logic is kept in *expenses.py* and the Budget logic is kept in *budgets.py*. Both use *database.py* to connect to the same SQLite database. Flask then sends the data to the HTML templates so it can be displayed in the browser.

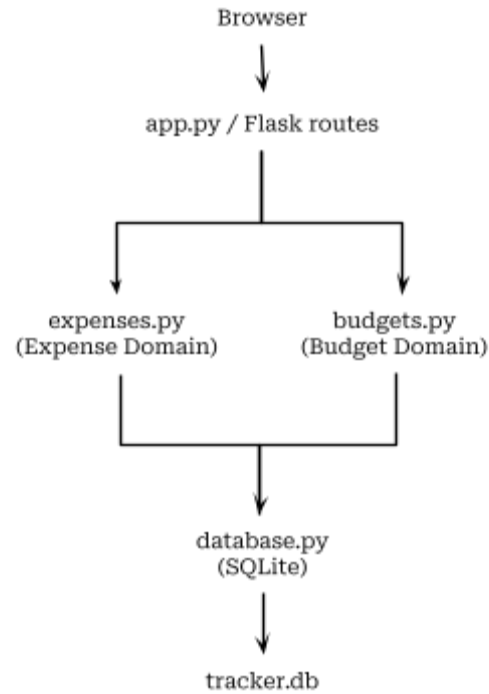


Figure 1: Application architecture. The Flask application handles requests from the browser and uses the Expense and Budget domains for the main application logic. Both domains use the same SQLite database.

5. Database design

The application uses one SQLite database with two tables: *expenses* and *budgets*. I kept them as separate tables because they represent different types of data. An expense represents money that has already been spent, while a budget represents the amount the user plans to spend for a category during a month. The Budget feature compares these two types of data when calculating how much money is left

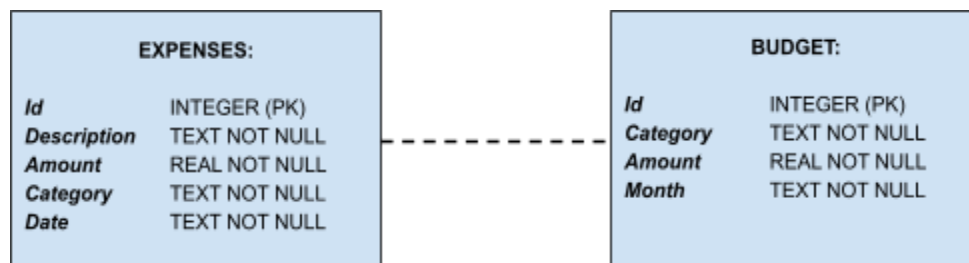


Figure 2: SQLite database schema. The database contains separate tables for expenses and budgets. There is no foreign key between them. When calculating spending, the application matches expenses to a budget using the category and month. The *UNIQUE(category, month)* rule prevents more than one budget from being stored for the same category and month.

6. Testing

I used pytest to test the main logic of the application. I created tests for both the Expense and Budget domains instead of only testing the web pages. This helped me check that the main functions work correctly without having to test everything manually through the browser.

The Expense tests check that invalid amounts are rejected, valid expenses are accepted, and expenses can be saved and retrieved from the SQLite database. The Budget tests check budget validation, saving budgets, calculating the amount spent from matching expenses, and calculating the remaining budget.

For database tests, I used *tmp_path* and *monkeypatch* so that the tests use a temporary database instead of changing the real application database.

I used pytest-cov to measure test coverage. The final test run had **8 passing tests** and **92% total coverage**, which is above the required 70%.

Command that was used: `..\venv\Scripts\python.exe -m pytest --cov=expenses --cov=budgets --cov=database --cov-report=term-missing`

Module	Coverage
expenses.py	86%
budgets.py	92%
database.py	100%
TOTAL	92%

7. Design Decisions and Limitations

I made several choices to keep the application simple while still meeting the requirements. I chose Flask instead of a larger framework such as Django because the application is small and does not need many built-in features. I also separated the Expense and Budget logic into different files instead of putting all the code inside `app.py`. This makes the code easier to understand and gives the two domains clearer responsibilities.

I chose SQLite because the application only needs a small local database. Expenses and budgets are stored in separate tables because they represent different types of information. The remaining budget is calculated when the budget page is loaded instead of being stored in the database. This avoids storing a value that could become incorrect whenever a new expense is added.

The application also has some limitations. It does not have user accounts or authentication, so it currently works as a single-user application. Categories are entered as text, which means differently written versions of the same category could be treated separately. The interface is also simple because I focused mainly on the backend functionality. These are features that could be improved in a future version.

8. AI Usage and Conclusion

I used ChatGPT during the development of this project to help with planning, understanding Flask and SQLite, debugging problems, writing tests, and improving the documentation. I did not accept every suggestion automatically. I tested the code as I developed the application and made changes based on the problems I found. The main AI-assisted steps are recorded in `AI_USAGE.md`, including what was suggested, what I used, and my own explanation of how the code works.

Overall, the final application meets the main goals I set at the beginning of the project. A user can add and view expenses, create monthly budgets, and see how much has been spent and how much remains. The Expense and Budget features are separated into their own domains while sharing the same SQLite database. The application also has automated tests with 92% coverage and is configured so that it can be prepared for deployment in the next assignment.

The project helped me understand how the different parts of a web application connect together, especially how the browser sends requests to Flask, how Flask uses Python functions to work with the database, and how the results are passed back to HTML templates.